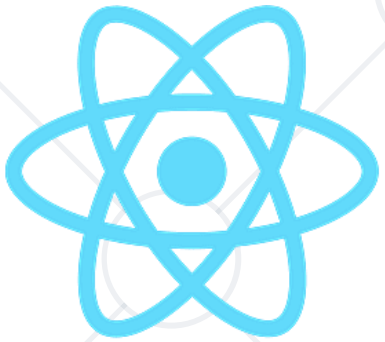


Advanced Styling and Basic Animations



React Native

SoftUni Team
Technical Trainers



SoftUni



Software University

<https://softuni.bg>

Table of Contents

1. Theming and Dark Mode
2. Gestures
3. Animations
4. Offline Mode
5. Unit Testing





Theming and Dark Mode

What Is Theming?

- 
- Theming is the ability to **control the visual appearance** of an app from a centralized configuration
 - Colors
 - Typography
 - Spacing
 - Shadows
 - Component styles
 - Brand identity

Theming separates design decisions from business logic.

Why Theming Matters

- 
- Without theming, styling becomes **unmanageable**
 - Hardcoded colors everywhere
 - Inconsistent UI
 - Painful redesigns
 - Poor accessibility
 - No dark mode support

Theming enables scalability and consistency.

- A theme is usually a **plain JavaScript object**
 - Centralized styles
 - Shared across the app
 - Easy to update
 - Predictable structure
 - Supports multiple variants

```
const theme = {colors:{background:'#fff',text:'#000',primary:'#4f46e5'}};
```

Themes are data, not magic.

- Dark mode is just **another theme**
 - Same structure
 - Different values
 - Switchable at runtime
 - User- or system-controlled
 - Improves accessibility and battery life

```
const darkTheme = {colors: {background: '#000', text: '#fff'}};
```

Dark mode is a UX feature, not a gimmick.

- Themes are **global** by nature
 - Use **Context API**
 - Wrap the app
 - Provide current theme
 - Allow switching
 - Avoid prop drilling

Theming is a perfect use case for Context.

- Navigation must follow the **same theme** as the rest of the app
 - Headers, tabs, drawers must adapt
 - Colors come from the theme object
 - Switching theme should update navigation instantly
 - Avoid hardcoded styles
 - Keep brand consistency across screens

```
<NavigationContainer theme={theme}>  
  {routes}  
</NavigationContainer>
```

If navigation ignores theming, the app feels broken.

- React Native exposes system **preferences**
 - Uses **useColorScheme**
 - Detects **light / dark**
 - Reacts to system changes
 - Platform-native behavior

```
const scheme = useColorScheme();
```

Follow the platform unless you have a reason not to.



Gestures

What Are Gestures?

- Gestures are **touch-based user interactions**
 - Translate **touch** into intent
 - Drive navigation and actions
 - Replace many traditional **buttons**
 - Must feel natural and responsive
 - Are **core** to mobile UX



Mobile UX is gesture-driven.

Why Gestures Need Special Handling

- **Gestures** are not simple clicks
 - Continuous input
 - Multiple fingers
 - Velocity & direction
 - Native performance required
 - Compete with scrolling

Gestures require native-level precision.

- React Native **Gesture Handler** is the standard solution
 - Native-backed gestures
 - Better performance
 - Declarative API
 - Works with animations
 - Industry standard

Never reimplement gestures manually.

- Gestures are defined **declaratively**
 - Clear intent
 - Predictable behavior
 - Easy composition
 - Native execution

Declarative gestures scale better.

- Different gestures communicate **different intentions**
 - **Tap** → trigger an action
 - **Long press** → reveal more options
 - **Swipe** → navigate or dismiss
 - **Drag** → move elements
 - **Pinch / rotate** → manipulate content

Choosing the right gesture is a design decision, not just an implementation detail.

- Gestures often **compete**
 - Scroll vs swipe
 - Parent vs child
 - Horizontal vs vertical
 - Multiple handlers
- Solutions:
 - Gesture Composition
 - Gesture Interaction

Gesture conflicts are design problems, not bugs.



Animations

Why Animations Matter

- 
- Animations provide **context and feedback**
 - **Improve** perceived performance
 - **Guide** user attention
 - **Communicate** state changes
 - **Reduce** cognitive load
 - Make apps feel **alive**

Motion is part of UX, not decoration.

- React Native provides a **basic animation system**
 - Declarative animations
 - Native driver support
 - Timing & spring animations
 - Good for simple cases

```
Animated.timing(value, {toValue:1, duration:300, useNativeDriver:true});
```

Animated API is powerful—but verbose.

- LayoutAnimation animates **layout changes automatically**
 - Handles size & position transitions
 - Triggered when the view hierarchy updates
 - Minimal code required
 - Great for inserts / removals
 - Native-driven for smoothness

```
LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
```

Change the layout → animation happens for free.

- **Reanimated** is the modern standard
 - Runs animations on UI thread
 - No JS thread blocking
 - Gesture + animation integration
 - Shared values
 - Worklets

Reanimated enables fluid, native-feeling motion.

- Shared values drive animations
 - Mutable values
 - Live on UI thread
 - Trigger reanimations
 - Used in styles

```
const progress = useSharedValue(0);
```

Animations are state machines.

- Styles react to shared values

```
const style = useAnimatedStyle(() => ({opacity: progress.value, }));
```

- Declarative
- Reactive
- Performant

UI reacts to motion state.



Offline Mode

Why Offline Mode Matters

- Network is unreliable
 - Poor connectivity
 - Airplane mode
 - Server downtime
 - Expensive data
 - User frustration



Offline-first is user-first.

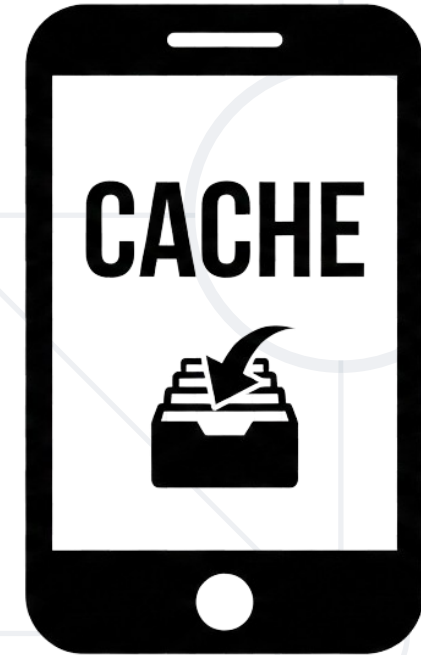
- Different levels of **offline behavior**
 - Read-only cache
 - Write queueing
 - Full offline mode
 - Sync on reconnect

Offline is a spectrum, not a switch.

- React Native can detect **connectivity**
 - Online / Offline
 - Changes over time
 - Platform-native APIs
 - Essential for UX

Apps must react to connectivity changes.

- Offline apps rely on cache.
 - Store API responses
 - Persist critical data
 - Load instantly
 - Sync later



Cache is a performance feature.

- Update UI **before** server response
 - Faster UX
 - Feels responsive
 - Requires rollback handling
 - Common in offline-first apps

Optimism improves perceived speed.

- **Always** show offline state
 - **Disable** destructive actions
 - Queue writes **safely**
 - Sync **transparently**
 - **Never** lose user data



Offline bugs destroy trust.



Unit Testing

What Is Unit Testing?

- Unit testing verifies **small, isolated pieces of logic**
 - Functions
 - Components
 - Hooks
 - Reducers
 - Utilities



Tests protect behavior, not implementation.

Why Testing Matters

- 
- Without tests:
 - Bugs regress silently
 - Refactoring is risky
 - Features break unexpectedly
 - Confidence drops
 - Velocity slows

Tests enable speed, not slow it down.

- React Native uses **testing libraries**
 - Render components
 - Query UI
 - Simulate interactions
 - Assert behavior

Test what users see and do.

- Focus on behavior.
 - Internal state ❌
 - Implementation details ❌
 - Rendered output ✅
 - User interactions ✅
 - Side effects ✅

Test from the user's perspective.

- Tests must be isolated
 - Mock APIs
 - Mock storage
 - Mock navigation
 - Mock timers

Isolation creates reliable tests.

What Should Be Tested

- Prioritize **critical** paths
 - Business logic
 - Reducers
 - Hooks
 - Forms
 - Error states

Test what would hurt if broken.

- Keep tests small
- Name tests clearly
- Avoid brittle assertions
- Test edge cases
- Run tests frequently

Tests are part of the codebase.

- **Theming & Dark Mode:** Centralizes styles (colors, typography) via a theme object and Context API to ensure UI consistency and seamless switching between light and dark modes.
- **Gestures & Animations:** Uses native-driven libraries like Gesture Handler and Reanimated to create responsive, high-performance interactions that make the app feel fluid and intuitive.
- **Offline Capability:** Prioritizes the user experience during poor connectivity by implementing data caching, network detection, and optimistic UI updates to prevent data loss.
- **Unit Testing:** Focuses on verifying isolated logic and user behavior



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

